

Blind analysis validation report: stripped OpenSSL command-line binary

What this document is

This is a full record of a blind reverse-engineering exercise run with the Vivarium tools against a stripped, statically linked build of the OpenSSL command-line program, followed by a verification of every conclusion against the original source code. It is written so that a reviewer can re-run the process, check each conclusion independently, and judge whether the result is strong enough to promote into the project's test fixtures.

The exercise was run genuinely blind. The source code was downloaded and set aside in a separate folder that was not opened until every conclusion below had already been recorded. The comparison and scoring sections were filled in last.

A short reading note for non-specialists: a "stripped" binary has had all of its human-readable names removed. There are no function names, no variable names, and no debug symbols. The analyst sees numbered placeholders like `FUN_0043f5c0` and must work out what each piece of code does from its behavior, the fixed data it contains, and the text strings it references.

Executive summary

- Subject: OpenSSL 4.0.1 command-line tool, statically linked, stripped, 7.9 MiB.
- Fifteen high-signal functions and constants were identified blind, then checked against the OpenSSL 4.0.1 source.
- Result: 15 of 15 identifications confirmed. Identification accuracy 100 percent.
- Mean confidence 92.3 out of 100. No high-confidence identification was refuted. The confidence estimates were, if anything, slightly conservative.
- Program-level findings (function count, call-graph shape, crypto presence, no indicators of malicious behavior) all matched the known nature of OpenSSL.
- Recommendation: the subject meets the fixture-promotion acceptance criteria defined in this report and is recommended for promotion to a test fixture. Promotion is a gated action requiring human approval. If rejected, the `samples/openssl-blind-analysis` directory should be discarded.

This is the second subject run through this process. The first was a stripped SQLite shell (see `docs/examples/blind-analysis-sqlite.md`). The two together provide an initial calibration of the method across very different programs: a single-purpose database engine and a large multi-command cryptography toolkit.

1. Subject under test (ingestion record)

Property	Value
File name (as imported)	openssl.blind
Program	OpenSSL command-line tool
Version (revealed at verification)	4.0.1
Size	7.9 MiB (8,316,656 bytes)
SHA-256	fba4556e7bba19522230cd0aab531d9cb380e6e6ebc0dc3a79defefadcb83060
Format	ELF 64-bit, x86-64, statically linked
Symbols	Stripped (no names, no debug info)
Build	gcc; ./Configure no-shared no-docs no-tests -static linux-x86_64; then strip -strip-all
Source tarball SHA-256	2db3f3a0d6ea4b59e1f094ace2c8cd536dfffb87cdc39084c5afa1e6f7f37dd09

The binary was built to resemble something found in the field: release build, fully static so the C library is baked in, and all names stripped. It lands at 7.9 MiB, inside the requested 5 to 10 MB range, which makes it a useful larger-scale counterpart to the 2.6 MiB SQLite subject.

At the time of analysis the analyst knew only the file on disk. The name `openssl.blind` was assigned by the person preparing the test and was treated as meaningless.

2. Process and methodology

2.1 The read-only tool chain

Vivarium runs Ghidra inside an isolated, locked-down worker container. The server process never loads the binary itself. Nothing about the binary is executed at any point. The analysis followed a fixed sequence of read-only tool calls:

1. `session_create` opens an isolated session.
2. `session_import` loads the file and verifies its SHA-256.
3. `session_analyze` runs Ghidra auto-analysis (disassembly, function discovery, cross-references, decompilation groundwork).
4. Program-level reads: `program_summary`, `coverage`, `call_graph_metrics`, `crypto_constant_scan`, `ioc_scan`.
5. Function-level reads: `function_context` (callees, callers, referenced strings) and `decompile_function` on the selected targets.
6. `session_close` tears down the worker and wipes its project store.

Every byte the tools return about the binary is treated as untrusted data. It is never executed, never rendered as markup, and any path or URL found inside it is never followed.

2.2 The blind protocol

Function selection was driven by the call graph, not by any prior knowledge:

- The highest fan-in functions (most called) were examined first. A function called from hundreds or thousands of places is almost always a core utility such as memory management, error

handling, or a container primitive.

- The highest fan-out functions (those that call the most others) were examined next. A function that calls very many others is usually a top-level dispatcher, such as a subcommand entry point.

Each function was named using only three kinds of on-binary evidence: its decompiled logic, the fixed data it embeds, and the text strings it references. The source folder remained closed throughout. Conclusions were written down and frozen before verification began.

3. What the tools returned (program level)

3.1 Program summary

Measure	Value	Plain meaning
Functions	17,223	A large program.
Imports	0	Nothing loaded from outside. Confirms static linking.
Exports	1	One entry point. An application, not a shared library.
Strings	18,048	A large body of embedded text to mine for clues.
Entry point	0x402400	Where execution begins.

3.2 Coverage

Measure	Value
Code ratio	0.649
Data ratio	0.174
Undefined bytes	1,480,415

About 65 percent of the analyzed space is recognized code and 17 percent is data. The remaining undefined region is normal for a large static binary that bundles a full C library and large constant tables.

3.3 Call graph

The call-graph metrics were computed over a 10,000-node sample of the 17,223 functions: 38,203 edges, 2,864 leaf functions, 4,728 root functions, and 31 recursive components. The most-called and most-calling functions from this sample became the identification targets in section 5.

3.4 Cryptographic constants

The constant scanner reported AES and MD5:

- Four AES lookup tables (S-box / T-tables) at 0x00a30d80, 0x00a30e80, 0x00a30f80, 0x00a31080.
- One MD5 initialization vector at 0x009fb310.

3.5 Indicators of compromise

The IOC scan returned 17 low-level matches, all benign and most of them false positives. This is itself an important result for an accurate review, so it is analyzed in detail in section 6.

4. Scoring methodology

This section defines how each identification was scored and how the aggregate metrics were computed. The goal is that a reviewer can reproduce the numbers, not take them on trust.

4.1 Evidence taxonomy

Each identification rests on one or more of the following kinds of evidence, listed roughly from strongest to weakest:

- E1, Embedded source path. The binary contains a literal source file path, for example `apps/s_client.c` or `crypto/err/err_local.h`. Release builds of many C projects embed `__FILE__` in assertions and error records. A path is close to a direct label and is the strongest single signal.
- E2, Distinctive string fingerprint. A set of error messages or option names so specific that they belong to exactly one function, for example the full `-connect / -proxy / -servername` option-error vocabulary of `s_client`.
- E3, Behavioral signature. A structural pattern in the decompiled logic that is characteristic of one routine, for example the pluggable-allocator pattern (check an overridable function pointer, otherwise call the default), or a zeroing allocator (allocate then `memset` zero), or a stack accessor (return the count, or minus one when the pointer is null).
- E4, Call-graph position. The function's fan-in or fan-out rank and its clustering with siblings, for example three adjacent functions each called about two thousand times forming the error-raising trio.
- E5, Constant fingerprint. A fixed data pattern that identifies an algorithm, for example an AES S-box table or an MD5 initialization vector.

4.2 Confidence rubric

Each identification was assigned a confidence score from 0 to 100, recorded before verification, using this rubric:

- 95 to 100: an embedded source path (E1) is present, or a unique behavioral signature (E3) is corroborated by at least one other independent evidence type.
- 85 to 94: a distinctive string fingerprint (E2) or a clear behavioral signature (E3), with supporting call-graph position (E4), but without a path string.
- 70 to 84: a single strong signal with limited corroboration, or a constant fingerprint (E5) for an algorithm family whose members share constants.
- Below 70: a single weak or ambiguous signal. None of the fifteen fell here.

Confidence measures how sure the analyst was before checking the source. It is deliberately separate from the verification outcome, so that the two can be compared to test calibration (section 4.4).

4.3 Verification outcomes

After conclusions were frozen, each was checked against the OpenSSL 4.0.1 source and assigned one of:

- CONFIRMED: the source symbol and signature match the blind identification, including the function's role.
- PARTIAL: the general role is correct but the specific name or boundary is off.
- REFUTED: the identification is wrong.

Verification used a signature search for the named symbol plus, where a path or unique string was the basis, a check that the string occurs in the named source file. All commands and their output are reproducible from section 8.

4.4 Aggregate metrics

- Identification accuracy = confirmed divided by total.
- Mean confidence = arithmetic mean of the per-identification confidence scores.
- Calibration check = comparison of confidence against outcome. Over-confidence (high confidence, refuted) is the failure mode that matters most for a tool whose output feeds an automated pipeline, so it is called out explicitly.
- False-positive accounting for the automated scanners (crypto and IOC), reported separately from function identification because those scanners are heuristic by design.

5. The fifteen identifications

Each row is a function or constant that Vivarium presented only as a numbered placeholder, the identification made blind, the source symbol and signature confirmed afterward, and the confidence and verification outcome.

#	Decompiled (address)	Surfaced as	Blind identification	Source symbol	Location	Conf.	Verdict
1	FUN_005c31c0	fan-in 2194	record file/line/func into the error state	ERR_set_debug	crypto/err/err_blocks.c:27	97	CONFIRMED
2	FUN_005c30a0	fan-in 2193	clear and advance the top error slot	ERR_new	crypto/err/err_blocks.c:14	88	CONFIRMED
3	FUN_005c32e0	fan-in 1921	record library id and reason code	ERR_set_error	crypto/err/err_blocks.c:38	85	CONFIRMED
4	FUN_00605a50	fan-in 1151	free with pluggable allocator	CRYPTO_free	crypto/mem.c:323	95	CONFIRMED
5	FUN_00605a70	fan-in 312	malloc with pluggable allocator	CRYPTO_malloc	crypto/mem.c:189	95	CONFIRMED
6	FUN_00605c60	fan-in 342	zeroing malloc (malloc then memset)	CRYPTO_zalloc	crypto/mem.c:222	96	CONFIRMED
7	FUN_005bfa90	ERR trio callee	return the per-thread error state	ERR_get_state	crypto/err/err.c	80	CONFIRMED
8	FUN_00652bd0	fan-in 503	stack accessor: count, or -1 if null	OPENSSL_sk_num	crypto/stack/stack.c:482	90	CONFIRMED
9	FUN_0043f5c0	fan-out 181, root	openssl s_client entry point	s_client_main	apps/s_client.c:980	99	CONFIRMED
10	FUN_004387f0	fan-out 133, root	openssl req entry point	req_main	apps/req.c:275	99	CONFIRMED
11	FUN_004656e0	fan-out 149, root	openssl x509 entry point	x509_main	apps/x509.c:367	92	CONFIRMED
12	FUN_00455bf0	fan-out 145, root	openssl speed entry point	speed_main	apps/speed.c:1816	99	CONFIRMED
13	FUN_00406be0	fan-out 130, root	openssl ca entry point	ca_main	apps/ca.c:303	99	CONFIRMED
14	tables at 0xa30d80	crypto scan	AES lookup tables	Te0..Te3	crypto/aes/aes_core.c:706	90	CONFIRMED
15	IV at 0x9fb310	crypto scan	MD5 initialization vector	MD5_Init	crypto/md5/md5_dgst.c:24	80	CONFIRMED

5.1 Representative walkthroughs

The memory and error core (rows 1 to 7). The most-called functions in the program were not the cryptography routines but the memory allocator and the error subsystem, which is exactly what one expects from a library where every operation can fail and must record why. `CRYPTO_free` (row 4) checks an overridable global function pointer and, if it has not been replaced, calls the default that

wraps the system free. `CRYPTO_malloc` (row 5) mirrors this and, on allocation failure, raises an error using three helper functions in sequence. Those three helpers are rows 1, 2, and 3: clear the top error slot, record the file, line, and function, then record the library identifier and reason code. The library identifier passed in is 15, which the source confirms is the constant for the crypto library. The decompiled body of `ERR_set_debug` matched the source helper line for line: free the previous file string, duplicate the new one with a `strlen-plus-one` allocation and a string copy, store the line number, then do the same for the function name.

The subcommand entry points (rows 9 to 13). Five of the fifteen were top-level command handlers, and these were the easiest to name because release builds embed the source file path in their error records. Seeing `apps/s_client.c` together with `localhost:4433` and the full `-connect / -proxy / -servername` option vocabulary is close to reading the label directly. The one command in this group without a visible path string in the sampled output, `x509`, was still named with high confidence from its distinctive `Serial number supplied twice` message, which verification located in `apps/x509.c`.

The cryptographic constants (rows 14 and 15) and a useful contrast. The scanner flagged four AES tables and one MD5 initialization vector. Both were correct. This is worth comparing to the earlier SQLite subject, where the same scanner reported MD5 but the constants actually belonged to SHA-1 (the two algorithms share their first four initialization words). In OpenSSL the MD5 label is correct, because OpenSSL genuinely contains MD5 in `crypto/md5/md5_dgst.c`. SHA-1 is also present and shares those four words, so the constant fingerprint alone cannot distinguish them. This is why row 15 carries a confidence of 80 rather than 95: the constant is a true positive for the family, and source inspection is needed to confirm which specific member is present. The lesson, consistent across both subjects, is that the crypto scanner is a reliable lead generator but the precise algorithm name should be confirmed by inspection.

6. False-positive analysis (scanners)

The automated crypto and IOC scanners are heuristic. Reporting their false-positive behavior is part of an honest validation, because a reviewer needs to know which scanner outputs are leads and which are conclusions.

Crypto scanner. Both reported algorithms (AES, MD5) are true positives. The only nuance, carried from the SQLite subject, is that an initialization-vector match identifies a hash family, not a unique algorithm, because MD5 and SHA-1 share their first four words. Precise naming needs inspection.

IOC scanner. All 17 matches are benign and the majority are false positives caused by ordinary OpenSSL data being shaped like network indicators:

- The values reported as IPv4 addresses (`1.3.101.112`, `1.3.101.110`, `1.3.101.111`, `1.3.101.113`, `1.3.14.3`, `1.101.3.4`) are ASN.1 object identifiers in dotted form. `1.3.101.112` is Ed25519, `1.3.101.110` is X25519, `1.3.101.111` is X448, `1.3.101.113` is Ed448, and `1.3.14.3` is an OIW branch. They are dotted-decimal data that happens to match an IPv4 pattern.
- The values reported as IPv6 (`00:00:00`, `23:59:59`) are time-of-day strings used in ASN.1 time formatting.
- The URL and domain matches (`https://github.com/dot-asm`, `mail.example.com`, format strings such as `http://%s`) come from an assembly attribution comment, example-certificate data, and URL-building format strings.
- The single email (`ke1d@dkuug.dk`) is a maintainer reference embedded in the ISO 8859 character-set tables.

The correct reading is that the IOC scanner found no evidence of malicious or network-callout behavior, and that several of its matches are structural false positives that a reviewer should expect from a cryptography toolkit. This does not reflect a defect in the binary or the tool; it reflects that

benign data can match indicator patterns, which is exactly why these outputs are leads rather than verdicts.

7. Metrics summary

Metric	Value
Identifications attempted	15
Confirmed	15
Partial	0
Refuted	0
Identification accuracy	100 percent
Mean confidence	92.3 / 100
Lowest confidence among confirmed	80 (ERR_get_state, MD5)
High-confidence refutations	0
Crypto scanner true positives	2 of 2 (AES, MD5)
IOC scanner malicious findings	0

Calibration. Every identification was confirmed, including the two scored at 80. There was no case of high confidence followed by refutation. If anything the confidence scores were slightly conservative, which is the safe direction for a tool whose output may feed an automated pipeline.

8. Reproducibility

Building the subject. Run `build-openssl-fixture.sh` in this directory. It downloads OpenSSL 4.0.1, verifies the source tarball hash, builds the static stripped binary, and checks the result against the recorded binary hash. Exact byte reproducibility depends on the local toolchain (gcc, binutils, libc); if the toolchain differs the layout may shift, in which case the analysis should be re-run before re-recording the hash.

Running the analysis. Import the binary into Vivarium and call, in order: `session_create`, `session_import`, `session_analyze`, then `program_summary`, `coverage`, `call_graph_metrics`, `crypto_constant_scan`, `ioc_scan`, then `function_context` and `decompile_function` on the addresses listed in `expected-analysis.json`, then `session_close`. The addresses are stable for this exact binary hash.

Verifying the identifications. With the OpenSSL 4.0.1 source extracted, the fifteen symbols and signatures in `expected-analysis.json` can be located with a signature search in the named files. The error trio is in `crypto/err/err_blocks.c`; the allocator is in `crypto/mem.c`; the stack accessor is in `crypto/stack/stack.c`; the five subcommands are in their respective `apps/*.c` files; AES tables are in `crypto/aes/aes_core.c`; the MD5 vector is in `crypto/md5/md5_dgst.c`.

9. Fixture-promotion assessment

The user asked that this subject be considered for promotion to a test fixture, based on its verification and confidence scoring, and discarded if rejected. The acceptance criteria and the evaluation against them are recorded in machine form in `expected-analysis.json` and summarized here.

Criterion	Threshold	This subject	Met
Identification accuracy	at least 0.90	1.00	yes
Mean confidence	at least 0.85	0.923	yes
High-confidence refutations	0	0	yes
Reproducible build	required	yes (build script + pinned hashes)	yes
Benign subject	required	yes (OpenSSL, no malicious indicators)	yes
Binary kept out of git history	required	yes (git-ignored; rebuilt on demand)	yes

All criteria are met, so the recommendation is to promote. Two points matter for how promotion is actually done:

- The 7.9 MiB binary is not committed. Repository policy keeps real binary samples out of git history and CI, and prefers synthetic fixtures. A promoted fixture therefore consists of the golden file (`expected-analysis.json`), the build script, and this report. The binary is regenerated on demand.
- A continuous-integration test built on this fixture is integration-tier, not a unit test, because it needs the Ghidra worker to produce the analysis that is then compared against the golden file. It should be gated accordingly and not added to the fast unit suite.

Promotion is a gated action. This report and its artifacts are the material needed for a human to make that decision. If the decision is to reject, discard the entire `samples/openssl-blind-analysis` directory; nothing else in the repository depends on it.

10. Limitations and threats to validity

- Selection. The fifteen targets were the highest-signal functions by call-graph position plus the crypto constants. They are not a random sample of the 17,223 functions, so the 100 percent figure is the accuracy on the most identifiable functions, not on an arbitrary one. This is the right population for a capability demonstration and a fixture, but it is not a claim about every function in the binary.
- Decompiler dependence. The identifications rest on Ghidra's decompilation and string cross-referencing. A decompiler error could in principle mislead an identification; here the source verification is the backstop that would have caught it.
- Toolchain reproducibility. The exact binary hash depends on the build toolchain. The analysis conclusions (which functions exist and what they do) are stable across toolchains, but the addresses and the hash are not.
- Single analyst, single tool version. These results reflect one run with one version of the tools. The fixture exists partly to detect drift in future versions.

Appendix A: raw program-level figures

- SHA-256: `fba4556e7bba19522230cd0aab531d9cb380e6e6ebc0dc3a79defefadcb83060`
- Size: 8,316,656 bytes
- Functions: 17,223; imports: 0; exports: 1; strings: 18,048
- Entry point: `0x402400`
- Coverage: code 0.649, data 0.174, undefined 1,480,415 bytes
- Call graph (10,000-node sample): 38,203 edges, 2,864 leaf, 4,728 root, 31 recursive components
- Crypto constants: AES S-box tables at `0xa30d80` , `0xa30e80` , `0xa30f80` , `0xa31080` ; MD5 IV at `0x9fb310`

- IOC matches: 17, all benign (see section 6)